GPU-computing project

Mahmoud Becki - Nicodème Rouger

Molecular dynamics simulations

2025 - 2026

Contents

1 Introduction **2**
1.1 Maxwell-Boltzmann distribution . . . 2
1.2 Goal . . . 3

2 Physical problem and numerical method used **3**
2.1 System considered . . . 3
2.2 Equation . . . 3
2.3 Boundary conditions and initial state . . . 4
2.4 Numerical scheme . . . 4

3 Data analysis **4**
3.1 Statistical measurements . . . 4
3.2 Relevant quantities to consider on simulation results . . . 5
3.3 Numerical application: Argon gas . . . 5

4 Performances **5**
4.1 Reference time . . . 5
4.2 Parallelisation . . . 5
4.2.1 What to parallelise? . . . 5
4.2.2 How to parallelise space calculations . . . 5
4.2.3 Pseudo-code . . . 5
4.3 Measured speedup and interpretation . . . 6
4.3.1 Why parallelization improves performance . . . 6
4.3.2 GPU hardware parallelism . . . 7
4.3.3 Measured performance improvement . . . 7
4.3.4 Scalability and optimization . . . 8

5 Code documentation **8**
5.1 Sequential Molecular Dynamics Simulation (CPU) . . . 8
5.1.1 Parameters . . . 8
5.1.2 Helper functions . . . 8

5.1.3 Block-centric data structures . . . 9
5.1.4 Overlapped candidate list construction . . . 9
5.1.5 Home ownership list construction . . . 9
5.1.6 Force computation (block-centric) . . . 10
5.1.7 Diagnostics . . . 10
5.1.8 Main execution flow . . . 10
5.2 GPU Molecular Dynamics Simulation (CUDA) . . . 10
5.2.1 Build and execution . . . 11
5.2.2 Fixed parameters . . . 11
5.2.3 List construction (count → scan → scatter) . . . 11
5.2.4 Force kernel (block-centric) . . . 11
5.2.5 Time integration (velocity Verlet) . . . 12
5.2.6 Diagnostics and timing . . . 12

6 Physical results **12**
6.1 First implementation with 2 particles . . . 12
6.2 N particles with GPU . . . 12

7 Conclusion **13**

---

1 Introduction

1.1 Maxwell-Boltzmann distribution

Thermodynamics and statistical physics starts from some important suppositions:

- Microscopic properties (position and velocities of particles), are considered as **random variables**. In this context one can define a probability density F that define the probability to measure a given set of parameters of the system (if only we could measure it...). For a set of N particles the phase space is defined as $\{\vec{r_i}, \vec{v_i}\}_{i\in[\![0,N-1]\!]} \in \mathbb{R}^{6N}$. In this context the probability to find the global system between the given states $\{\vec{r_i}, \vec{v_i}\}_{i\in[\![0,N-1]\!]}$ and $\{\vec{r_i} + d\vec{r_i}, \vec{v_i} + d\vec{v_i}\}_{i\in[\![0,N-1]\!]}$ at a given time t is :

$$F(\{\vec{r_i}, \vec{v_i}\}_{i\in[\![0,N-1]\!]}, t)\Pi_{i=0}^{N-1} d\vec{r_i} d\vec{v_i}$$

- Distance between two interparticles collisions is much bigger than the range of interparticle forces. It means that there is no correlation between particules such as **the density function is separable**:

$$F(\{\vec{r_i}, \vec{v_i}\}_{i\in[\![0,N-1]\!]}, t) = \Pi_{i=0}^{N-1} F_0(\vec{r_i}, \vec{v_i}, t)$$

- At thermodynamic equilibrium F_0 do not depend on time, the 3 components of velocities are independant and the distribution of velocities is **isotropic** (do not depend on the direction considered).

Under these hypothesis the number of particles per volume unit which velocities are in an elementary phase volume d^3v centered on $\vec{v}$ is given by:

$$d^3n(\vec{v}) = nf(\vec{v})d^3v$$

where n is the density of particles and $f(\vec{v})$ is the **Maxwell-Boltzmann distribution**:

$$f(\vec{v}) = (\frac{m}{2\pi k_B T})^{3/2} exp(-\frac{mv^2}{2k_B T})$$

with $k_B = 1.38 \cdot 10^{-23} J.K^{-1}$ the Boltzmann constant, T the kinetic temperature and m the mass of each particle.

Isotropic hypothesis allow to integrate this formula over all direction. It leads to the distribution function of the velocity norm $v = |\vec{v}|$. Instead, the number of particles per volume unit which velocity norm is between v and $v + dv$ can be written:

$$dn(v) = nf(v)dv = 4\pi n(\frac{m}{2\pi k_B T})^{3/2} v^2 exp(-\frac{mv^2}{2k_B T})dv \tag{1}$$

1.2 Goal

The aim of this project is to verify the hypothesis mentioned above by modelizing a fluid with potential interaction between each particles.

Calculating tajectories of each particle allow to evaluate the function (1) and verify that the molecular model fit well with the statistical theory at thermodynamic equilibrium (when the simulation has converged).

Some extra consideration should be using the simulation to evaluate diffusion coefficients that are not predicted by statistical theory.

2 Physical problem and numerical method used

The model developped her is inspired from [2]. An other source of inspiration has been [1].

2.1 System considered

The system considered is a fluid composed of N particles in a box of volume V. Particles are considered as atoms of mass m that interact with a Lenard-Jones potential. One can define $(\vec{r_i})_{0 \leqslant i < N}$ the position of atoms in a Galilean reference frame and $r_{ij} = |\vec{r_{ij}}| = |\vec{r_j} - \vec{r_i}|$ the distance between 2 particles. The continuous Lenard-Jones potential between two particles that is 0 when $r_{ij} > r_c$ can be written:

$$u(r_{ij}) = \begin{cases} 4\epsilon \left[\left(\frac{\sigma}{r_{ij}}\right)^{12} - \left(\frac{\sigma}{r_{ij}}\right)^{6} \right] + \epsilon & \text{if } r_{ij} < r_C = 2^{1/6}\sigma \\ 0 & \text{else} \end{cases}$$

2.2 Equation

Energy of each atom:

$$\begin{aligned} e_i &= e_i^{(k)} + e_i^{(p)} \\ &= \frac{1}{2} m \vec{v_i}^2 + \sum_{j \neq i} u(r_{ij}) \end{aligned}$$

With the rescaling $r \to r\sigma$, $e \to e\epsilon$, $t \to t\sqrt{m\sigma^2/\epsilon}$, one can write the dimensionless Newton's second law as:

$$\frac{d^2 \vec{r_i}}{dt^2} = 48 \sum_{\substack{j \in [\![0, N-1]\!] \setminus \{i\} \\ r_{ij} < r_C}} \left(r_{ij}^{-14} - \frac{1}{2} r_{ij}^{-8} \right) \vec{r}_{ij}. \tag{2}$$

2.3 Boundary conditions and initial state

Boundary effects are not studied here. To describe a quasi infinte fluid, periodic boundary conditions are used.

A first simple initial state can be:

- Atoms positioned at the sites of a regular lattice,
- Initial velocities with a constant norm (depending on the temperature T) and with random orientation such as global velocity of the fluid is 0 (no global movement).

Others initial values can be considered in a second time.

2.4 Numerical scheme

The leapfrog method is adapted to this problem because it conserve the energy and provide a local error in $\mathcal{O}(\delta t^4)$ for positions and $\mathcal{O}(\delta t^2)$ for velocities.

Writting

$$\vec{A}(\vec{r_i}) = 48 \sum_{\substack{j \in [\![0,N-1]\!] \setminus \{i\} \\ r_{ij} < r_C}} (r_{ij}^{-14} - \frac{1}{2} r_{ij}^{-8}) \vec{r_{ij}}$$

The equation (2) become:

$$\frac{d^2 \vec{r_i}}{dt^2} = \vec{A}(\vec{r_i})$$

Then discretizing time and applying the leapfrog method lead to the scheme:

$$\begin{cases} \vec{v_i}(t + \delta t/2) & = \vec{v_i}(t - \delta t/2) + \vec{A}(\vec{r_i}(t))\delta t \\ \vec{r_i}(t + \delta t) & = \vec{r_i}(t) + \vec{v_i}(t + \delta t/2)\delta t \end{cases}$$

3 Data analysis

3.1 Statistical measurements

Under the rescaling given in 2.2 the temperature became $T \to T\epsilon/k_B$, the velocity $v \to v\sqrt{\frac{\epsilon}{m}}$ and the Maxwell-Boltzmann distribution:

$$dn(v) = nf(v)dv = n\sqrt{\frac{2}{\pi}} v^2 exp(-\frac{v^2}{2T})dv$$

One can also calculate the dimensionless kinetic and potential energy per atom as:

$$e_k = \frac{1}{2N} \sum_{i=0}^{N-1} \vec{v_i}^2, \; e_p = \frac{4}{N} \sum_{\substack{j \in [\![0,N-1]\!] \setminus \{i\} \\ r_{ij} < r_C}} (r_{ij}^{-12} - r_{ij}^{-6})$$

and the dimensionless temperature as:

$$T = \frac{1}{3N} \sum_{i=0}^{N-1} \vec{v_i}^2.$$

Finaly the Virial expression allow to calculate dimensionless microscopic pressure:

$$P = \frac{1}{3V} (\sum_{i=0}^{N-1} \vec{v_i}^2 + 48 \sum_{i<j} (r_{ij}^{-12} - \frac{1}{2} r_{ij}^{-6})).$$

Assuming ergodicity one can calculate thermodynamic quantites as time average of the microscopic quantities and study their standard deviation.

3.2 Relevant quantities to consider on simulation results

- Variance of the maxwell distribution (check it converge to $\sqrt{T}$)
- Mechanical energy (check it remain constant over all the computation)

3.3 Numerical application: Argon gas

Gas poperties:

$$\begin{aligned} \sigma =& 3.4\dot{A} \\ \frac{\epsilon}{k_B} =& 120K \\ m =& 6.6904 \cdot 10^{-26} kg \end{aligned}$$

Box length:

$$L_{box} = 1cm^3$$

4 Performances

4.1 Reference time

A first implementation of the calculations is done sequentialy using C++. This implementation give a reference of the time needed to solve the problem sequentialy.

4.2 Parallelisation

4.2.1 What to parallelise?

In a first evaluation we consider that the time evolution can't be parallelised. So we have to parallelise force calculation over space.

4.2.2 How to parallelise space calculations

Making cubes of a given size l_c ($l_c \gg r_c$) and calculating interaction forces inside each cube at the same time.

Problem at bounds: we need to exchange information about all particles that are separated by less than r_c of the considered particle: if this particle is close to a separation between two blocks it will require some exchange of information. Using overlapping seems to be a good idea (take particles that are outside the box but distant of less than r_c of the box).

Other question: when to update in which box is each particle? In the first article we read they update each time step maybe it's a lot? We need to test this. If we don't do it every timestep the simulation may miss some interactions.

Updating = Synchronisation

4.2.3 Pseudo-code

- define the block length: $l = \frac{L}{k} + 2r_c$ for k blocks overlapped
- define the blocks data: `blocks` an array of length k^3
- fill in block data (and put it in shared memory?)
  - for each particle j:

- for each dimention $\alpha \in \{x, y, z\}$:
  - Euclidian division: $r_\alpha^{(j)} = q\frac{L}{k} + p$
  - if $p \in [r_c, \frac{L}{k} - r_c]$: $n_{block,\alpha} = \{q\}$
  - elif $p \in [0, r_c]$: $n_{block,\alpha} = \{q, q-1\}$
  - elif $p \in [\frac{L}{k} - r_c, \frac{L}{k}]$: $n_{block,\alpha} = \{q, q+1\}$
- for m_x, m_y, m_z in $n_{block,x} * n_{block,y} * n_{block,z}$:
  - `blocks`$[m_x,m_y,m_z]$.append$(\vec{r}_j,\vec{v}_j)$

- execute parallel kernel: compute forces and parameters for the next time step in each block

- fill in block data an other time...

4.3 Measured speedup and interpretation

Molecular Dynamics simulations are dominated by the computational cost of **force evaluation**. At each time step, every particle accumulates contributions from neighboring particles within a cutoff radius, leading to a large and repetitive numerical workload.

In the baseline CPU implementation considered here, the solver is **single-threaded** (one CPU thread performs all work sequentially). In the GPU implementation (CUDA), the workload is distributed across many GPU threads that execute concurrently. The purpose of this section is to interpret the measured speedup while explicitly accounting for the fact that the CPU code uses **1 thread**, whereas the GPU executes **many threads in flight**.

4.3.1 Why parallelization improves performance

For a cutoff-based MD solver, each particle i evaluates:

$$\vec{a}_i = \sum_{j \in \mathcal{N}(i)} \vec{f}(\vec{r}_i - \vec{r}_j),$$

where $\mathcal{N}(i)$ denotes the set of neighbor candidates within the cutoff.

Once neighbor candidates are known, force accumulation for each particle can be performed independently (no cross-particle dependency is required in the force kernel because each thread writes only (a_x, a_y, a_z) for its own particle i). This makes the force step well suited to thread-level parallelism.

CPU (baseline): all interactions are computed by **one** instruction stream (one thread), so the effective parallelism is:

$$P_{\text{CPU}} = 1.$$

GPU (CUDA): the work is split across many threads. In the block-centric force kernel:

- Each CUDA block corresponds to a spatial cell b (`blockIdx.x = b`).

- Threads inside the block stride over the home-particle list of that cell:

```
hi = h_begin + threadIdx.x;   hi += blockDim.x.
```

For each active cell, up to **256 threads** may compute forces for different home particles concurrently (the force kernel is launched with `blockDim.x = 256`).

Across the whole grid, the total number of launched threads for the force kernel is:

$$N_{\text{threads, launched}} = n_b \times 256,$$

where n_b is the total number of cells (`nb`).

Not all launched threads run simultaneously. GPUs execute threads in groups of warps, and concurrency is bounded by hardware resources (SMs, warps/SM, registers, shared memory, etc.). A practical upper bound on concurrent threads is:

$$N_{\text{threads, concurrent}} \approx N_{\text{SM}} \times W_{\text{active/SM}} \times 32,$$

where N_{SM} is the number of streaming multiprocessors and $W_{\text{active/SM}}$ is the number of active warps per SM (limited by occupancy and kernel resource usage).

4.3.2 GPU hardware parallelism

The GPU used for the CUDA implementation has the following relevant properties:

- GPU: Quadro RTX 6000 (Turing), compute capability 7.5
- SM count: $N_{\text{SM}} = 72$
- Warp size: 32
- Max threads/block: 1024
- Shared memory / block: 48 KB
- Registers / block: 65536
- Global memory: 21 GB

This configuration enables large warp-level concurrency across 72 SMs and allows many threads to remain in flight to hide memory latency.

4.3.3 Measured performance improvement

For the tested configuration (approximately one million particles), the observed runtimes were:

- **CPU (single-thread, block-centric)**: ~ 1497 s
- **GPU (CUDA, block-centric)**: ~ 36.7 s

The end-to-end speedup is:

$$S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}} \approx \frac{1497}{36.7} \approx 40.8 \times .$$

Because the CPU baseline uses one thread, the measured speedup can be interpreted as an **effective throughput gain relative to 1-thread execution**. Defining an effective number of CPU-equivalent threads as:

$$P_{\text{eff}} \equiv S \times P_{\text{CPU}} = S,$$

gives:

$$P_{\text{eff}} \approx 40.8.$$

This does not mean that only ~ 41 GPU threads were used. The force kernel launches and schedules many thousands of threads; the achieved speedup is bounded by memory traffic, instruction mix, occupancy limits, and non-force overheads (list build, reductions/diagnostics, synchronization, etc.).

4.3.4 Scalability and optimization

Timing separates three main components:

- list construction (duplication + prefix sums + scatters),
- force kernel (dominant compute phase),
- diagnostics (per-particle energies + reductions + device-to-host copies at print steps).

Let p be the fraction of time spent in the highly parallel region (primarily forces) and $(1-p)$ the fraction spent in less scalable parts. A useful view is:

$$S = \frac{1}{(1-p) + \frac{p}{P_{\text{GPU,eff}}}},$$

where $P_{\text{GPU,eff}}$ is an effective parallelism term that absorbs GPU concurrency, memory bandwidth, and latency hiding.

Promising next gains are typically obtained by improving reuse in the force kernel (shared-memory tiling when justified) and reducing list/diagnostic overhead (rebuild lists less frequently using a Verlet skin, and reduce synchronization during print steps).

5 Code documentation

5.1 Sequential Molecular Dynamics Simulation (CPU)

The C++ sequential code of the simulation is a single-file implementation that runs a Molecular Dynamics simulation of N particles in a 3D cubic box with:

- Periodic Boundary Conditions (PBC)
- Lennard-Jones interactions (cutoff + shifted potential)
- Reduced units (all constants set to 1)

5.1.1 Parameters

- `dt (real)` → time step.
- `n_steps (int)` → number of simulation steps.
- `save_every (int)` → print diagnostics every `save_every` steps.

5.1.2 Helper functions

`int wrap_int(int q, int k)` Periodic wrapping for integer indices. Ensures `q` stays in $[0, k-1]$. Used to wrap neighboring block indices under periodic boundary conditions.

`int block_index(int bx, int by, int bz, int k)` Flattens a 3D block coordinate `(bx,by,bz)` into a 1D array index:

$$\texttt{index} = (bx \cdot k + by) \cdot k + bz.$$

`void wrap_position(real& x, real L)` Applies periodic wrapping to a floating-point coordinate `x` so it stays in $[0, L)$. Used after updating positions to enforce PBC.

`void minimum_image(real& dx, real& dy, real& dz, real L, real halfL)` Applies the minimum image convention to relative displacements `(dx,dy,dz)` so the shortest periodic displacement is used.

`int axis_blocks(real coord, real s, real rc, int kblocks, int out[2])` Returns the list of candidate block indices along one axis for a particle at coordinate `coord`. The function always outputs the home block in `out[0]`. If the particle is within `rc` of a block boundary, it also outputs the neighboring block in `out[1]`. The return value is either 1 or 2.

5.1.3 Block-centric data structures

The simulation uses two complementary block lists:

- Overlapped candidate list (many-to-many): each particle may appear in multiple blocks (up to $2 \times 2 \times 2 = 8$) so that interactions within a cutoff can be found by scanning only the current block.
- Home ownership list (one-to-one): each particle appears exactly once in its home block. This defines which block owns and updates that particle.

5.1.4 Overlapped candidate list construction

`void build_block_list(...)` Builds the overlapped candidate structure.
Outputs:

- `blockCount[b]`: number of particle insertions into block `b`.
- `blockStart`: prefix-sum boundaries (size `nb+1`).
- `blockList`: packed particle indices for all blocks.
- `particleBlock[i]`: home block id for each particle `i`.

Implementation steps:

1. For each particle, compute candidate blocks along each axis using `axis_blocks`. Insertions per particle are the product of axis choices (1 or 2 per axis).
2. Count insertions per block in `blockCount`.
3. Compute `blockStart` by prefix sum so each block `b` has range [`blockStart[b]`, `blockStart[b+1]`).
4. Fill `blockList` using write pointers initialized from `blockStart`.

5.1.5 Home ownership list construction

`void build_home_list(...)` Builds the ownership mapping.
Outputs:

- `homeCount[b]`: number of owned particles in block `b`.
- `homeStart`: prefix-sum boundaries (size `nb+1`).
- `homeList`: packed particle indices by home block.

Implementation steps:

1. Count owned particles per block using `particleBlock`.
2. Prefix sum to build `homeStart`.
3. Fill `homeList` so each block `b` has owned range [`homeStart[b]`, `homeStart[b+1]`).

5.1.6 Force computation (block-centric)

`void compute_forces_blockwise(...)` Computes accelerations `(ax,ay,az)` and the total potential energy using Lennard-Jones interactions with a cutoff.
Key rule:

- Particles are updated only by the block that owns them (from `homeList`).
- Each owned particle scans only the candidates stored in the same block (from `blockList`).

Potential shift:

$$U_c = 4\left(r_{\text{cut}}^{-12} - r_{\text{cut}}^{-6}\right),$$

and the code uses $U(r) - U_c$ so that $U(r_{\text{cut}}) = 0$.

5.1.7 Diagnostics

`compute_kinetic_momentum_temperature(...)` Computes total kinetic energy, instantaneous temperature $T = \frac{2K}{3N}$, and total momentum components by looping over particle velocities.

5.1.8 Main execution flow

```
initialize positions, velocities
remove center-of-mass velocity
set rCut, kblocks, s
build_block_list(...)
build_home_list(...)
compute_forces_blockwise(...)
init half-step velocities (vxh,vyh,vzh)

for each step:
    drift positions with half-step v
    wrap positions (PBC)
    build_block_list(...)
    build_home_list(...)
    compute_forces_blockwise(...)
    kick half-step velocities
    reconstruct v for diagnostics
    print diagnostics
```

5.2 GPU Molecular Dynamics Simulation (CUDA)

The file `md_cuda.cu` is a single-file CUDA implementation of a Molecular Dynamics simulation for N particles in a 3D cubic periodic box with:

- Periodic Boundary Conditions (PBC)
- Lennard–Jones interactions with WCA cutoff ($r_{\text{cut}} = 2^{1/6}$)

- Reduced units (all constants set to 1)

The GPU version follows the same CPU-style block-centric algorithm:

- duplicated candidate list (particles may appear in up to $2^3 = 8$ cells),
- home ownership list (each particle belongs to exactly one cell),
- block-centric force kernel: 1 CUDA CTA = 1 spatial cell.

5.2.1 Build and execution

Compilation:

```
module load cuda
nvcc -O3 -std=c++17 -lineinfo md_cuda.cu -o md_cuda
```

Run:

```
srun --gres=gpu:1 ./md_cuda
```

5.2.2 Fixed parameters

- `N_req = 1000000`, `density = 0.8`
- `dt = 0.001`, `n_steps = 1000`, `save_every = 10`
- `DUP_MAX = 8`

5.2.3 List construction (count → scan → scatter)

Duplicated candidate list:

- `count_blocks_dup_kernel` counts insertions (`atomicAdd` into `blockCount`).
- `cub::DeviceScan::ExclusiveSum` builds `blockStart`.
- `scatter_blocks_dup_kernel` fills `blockList` using per-cell write pointers.

Home ownership list:

- `count_home_kernel` counts ownership per cell.
- `cub::DeviceScan::ExclusiveSum` builds `homeStart`.
- `scatter_home_kernel` fills `homeList`.

5.2.4 Force kernel (block-centric)

`compute_forces_blockcentric_kernel` Maps one CUDA thread block to one spatial cell b.
Owned particles are read from [`homeStart[b]`, `homeStart[b+1]`) and candidates from [`blockStart[b]`, `blo`
Each CUDA thread processes a strided subset of owned particles:

$$hi = h_begin + \texttt{threadIdx.x} + t \cdot \texttt{blockDim.x}.$$

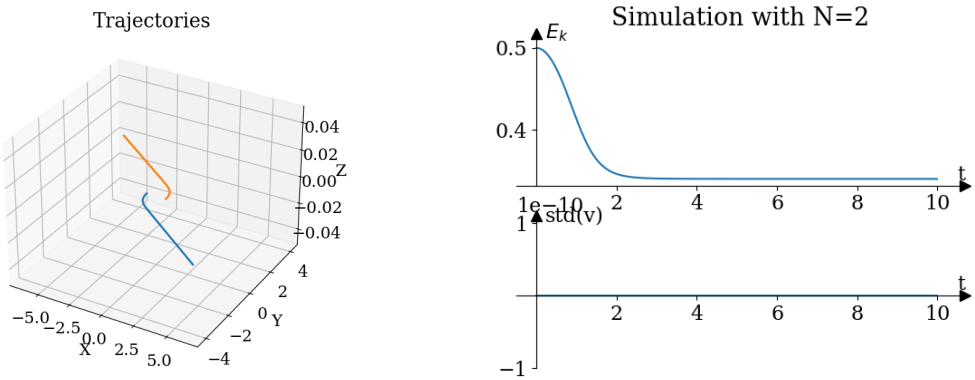

Figure 1: CPU simulation results for 2 particles. On the left: fiew trajectories. On the right: total kinetic energy and mean velocity standard deviation.

5.2.5 Time integration (velocity Verlet)

1. `copy_accel_kernel`
2. `verlet_position_kernel` (wrap)
3. list rebuild
4. `compute_forces_blockcentric_kernel`
5. `verlet_velocity_kernel`
6. diagnostics every `save_every` steps

5.2.6 Diagnostics and timing

Potential energy and kinetic energy/momentum are computed with per-particle kernels and reduced with CUB. Timing uses CUDA events and accumulates list build time, force time, and diagnostics time.

6 Physical results

6.1 First implementation with 2 particles

The figure 1 shows trajectories and kinetic energy for the well known case of 2 particles interaction with a given initial state. Standard deviation of the velocities is also plotted depending on simulation time. This first implementation were done to check that the simulation is giving coherent results. The two particles have the same velocity at each timestep so the standard deviation is zero. Kinetic energy remain constant over long time: particles are not interacting for long time. These results are consistent with analytical solution of two particles interaction. It validate our model and the kinetic energy calculations.

6.2 N particles with GPU

Same observables are plot figure 2. A good point is that kinetic energy and velocity standard deviation are converging. It means that thermodynamic observables such as temperature or energy are well defined. The physical comportement expected seems to be validated by our

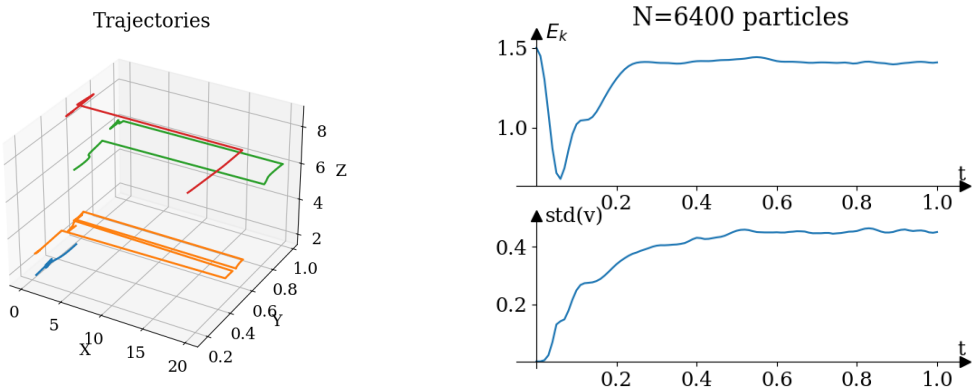

Figure 2: GPU simulation results. On the left: fiew trajectories. On the right: total kinetic energy and mean velocity standard deviation.

modelisation. An unexpected result is that trajectories plotted seem too show large straight trajectories. It means that particles are not interacting that much. Maybe making a simulation with a higer density could fix it.

7 Conclusion

GPU has been used to manage to simulate high number particle interaction to compare it with statistical physics. It allowed to learn not only new physical tools but also informatics issues (including parallelisation, memory managing, performances evaluation, data processing).

Physicaly the simulation fit well the expeceted behaviour of the considered system. Numericaly, this project allowed to show the advantages and disadvantages of GPU programming. In fact, we showed the time gain of parallelisation but we also experiment the difficulty to code and debug a GPU code.

References

[1] "Computational Soft Matter: From Synthetic Polymers to Proteins. 2: Lecture Notes". NIC Series volume 23. Jülich: NIC, 2004. ISBN: 978-3-00-012641-3.

[2] Dennis Rapaport. "The Art of Molecular Dynamics Simulation". *Cambridge University Press* (2004).